Networker Clone

Project Documentation — a full-stack corporate IT-services website

React + Vite + Tailwind CSS (client) | Node.js + Express + MongoDB (server)

1. What is this project?

This project is a **clone of a corporate IT-services company website** (brand shown in the code as "socialnetworker"; some file names such as *LifeAtXilligence.jsx* show the site was originally modelled on Xilligence). The company presents itself as a software and technology firm founded in 2000 that builds "made-to-order software teams that are exclusive to your company".

The website is a **marketing / brochure website**. Visitors can read about the company, browse services, industry verticals and technologies, see past work, view open jobs, and send an enquiry through a contact form. There is no user login and no admin panel — content is stored in the database (or in local fallback data) and the only data users send back is the contact form.

The project follows a **two-part architecture**: a React single-page application (frontend) that talks to a REST API built with Express and MongoDB (backend). A special design choice: if the backend is not running, the frontend automatically falls back to local mock data, so the UI always works.

2. Technology stack

Layer	Technology	Purpose
Frontend	React 18, Vite 5	SPA framework and fast dev/build tool
Styling	Tailwind CSS 3, PostCSS	Utility-first CSS; custom brand colours (navy #211E5B, orange #E1552D, purple #6E2FA8)
Routing	react-router-dom 6	Client-side page navigation (35+ routes)
HTTP	axios	API calls (base URL from VITE_API_URL, 60 s timeout)
Icons	react-icons	UI icons (Heroicons set)
Backend	Node.js (ES modules), Express 4	REST API server
Database	MongoDB + Mongoose 8	Stores all site content and contact messages
Security/Utils	helmet, cors, morgan, dotenv	HTTP headers, CORS, request logging, env config
Dev tools	nodemon, ESLint	Auto-restart server; linting

3. Folder structure

```
networker-clone/
|-- client/                       React frontend
|   |-- src/
|   |   |-- App.jsx               All routes
|   |   |-- main.jsx              App entry point
|   |   |-- components/
|   |   |   |-- layout/           Header, Footer, MobileMenu
|   |   |   |-- home/             Hero, ServicesSection, AboutSection,
|   |   |   |                     LocationsSection, ServiceCard
|   |   |   `-- common/           Button, Container, Logo, NavDropdown,
|   |   |                         ContentPage (generic page renderer)
|   |   |-- pages/                ~35 page components
|   |   |-- services/api.js       API layer + mock fallback
|   |   `-- data/mockData.js      Local fallback content (~880 lines)
|   |-- tailwind.config.js, vite.config.js
|   `-- public/images/            SVG/image assets
`-- server/                       Express backend
    `-- src/
        |-- server.js             Starts server after DB connect
        |-- app.js                Middleware + route mounting
        |-- config/db.js          MongoDB connection
        |-- models/               11 Mongoose schemas
        |-- controllers/          11 controllers
        |-- routes/               11 route files
        |-- middleware/           404 + error handler
        `-- data/seedData.js      Seeds MongoDB with content
```

4. Frontend in detail

4.1 Layout

- **Header** — sticky white bar with logo, hover dropdown menus (About, Services, Verticals), simple links (Technologies, Our Work, Careers) and a "Contact us" button. Adds a shadow on scroll. On mobile it becomes a hamburger **MobileMenu** that locks body scrolling while open.
- **Footer** — contact email and phone numbers (Canada, UAE, India), company/services/location links, social icons, copyright and disclaimer.
- **Home page** — Hero (navy gradient with headline and "Services" button), Services grid, About section, and a Locations map with markers for Canada (Toronto), UAE (Dubai), India (Trivandrum) and Singapore.

4.2 Pages and routes

Section	Routes
Home	/
About	/about, /about/company-overview, /about/executive-leadership, /about/client-partners, /about/news, /about/events, /about/life-at-socialnetworker
Services	/services, /services/mobile-apps, /services/cloud-web, /services/design-ui-ux, /services/ai-ml-bi, /services/bps, /services/product-engineering, /services/factoring-solutions
Quality Engineering	/services/quality-engineering and sub-pages: functional-testing, test-automation, non-functional, performance-testing, security-testing; plus /sap-testing
Verticals	/verticals/seffo, /verticals/e-commerce, /verticals/digital-health, /verticals/banking-financial-services
Other	/technologies, /our-work, /careers, /contact, /privacy-policy, /locations/:country, * (404 page)

4.3 Key frontend patterns

- **ContentPage component** — a generic renderer used by ~20 pages. Each page only supplies a *slug*; ContentPage fetches the matching content (title, intro, image, highlights, stats, sections with bullets) and handles loading, error and empty states.
- **API with silent fallback** — every getter in *services/api.js* (getServices, getLocations, getPageContent, getTeamMembers, getNewsPosts, getEvents, getWorkProjects, getJobPostings, …) tries the real API first and returns mock data if the call fails or returns nothing. Only *getJobPostings* falls back to an empty list, and the contact form submit has no fallback (it must reach the server).
- **Contact page** — form with name, email, phone, company, message; shows submitting / success / error states and also lists contact details and office locations.
- **Careers page** — shows benefits (insurance, medical camp, entertainment, flexible timing, onsite opportunities) and open job postings from the API.
- Components cancel stale requests with a *cancelled* flag in useEffect to avoid state updates after unmount.

5. Backend in detail

The server (*server.js*) connects to MongoDB first, then starts Express on the port from *.env* (default 5000). If MongoDB is unreachable the API still starts but data routes return errors. *app.js* applies helmet, CORS (any origin), JSON parsing and morgan logging, then mounts the routes below. Unknown routes return a JSON 404 and errors go through a central error handler.

Endpoint	Model	Description
GET /api/health	—	Health check: {status:"ok"}
GET /api/services, /:id	Service	Service cards (id, title, description, image, link, order)
/api/verticals	Vertical	Industry verticals
/api/locations	Location	Office locations with map position (top/left %) and colour
POST /api/contact	ContactMessage	Saves enquiry; requires name, email, message; status new/contacted/closed
GET /api/pages/:slug	PageContent	Generic page content: title, intro, image, highlights, sections, stats
/api/team	TeamMember	Leadership team
/api/partners?type=	Partner	Clients and partners (type = client | partner)
/api/news	NewsPost	News articles
/api/events	Event	Company events
/api/work-projects	WorkProject	Portfolio of past work
/api/jobs	JobPosting	Open positions (title, experience, location, details, isOpen)

Seeding: *npm run seed* runs *seedData.js*, which clears Service, Vertical, Location, PageContent, TeamMember, Partner, NewsPost, Event and WorkProject collections and re-inserts the content (same as the frontend mock data). Note: ContactMessage and JobPosting are not seeded.

6. Business content covered by the site

- **Product Engineering:** Mobile + Apps, Cloud + Web, Design + UI/UX, AI/ML + BI, BPS (business process services).
- **Quality Engineering:** functional, automation, non-functional, performance and security testing; SAP testing.
- **Factoring Solutions** (finance-related software service).
- **Verticals:** Seffo (own product), E-commerce, Digital Health, Banking & Financial Services.
- **Offices:** Toronto (Canada), Dubai (UAE), Trivandrum (India), Singapore.

7. How to run

Backend:

```
cd server
npm install
# create .env with PORT, MONGO_URI, CLIENT_ORIGIN
npm run seed     # load content into MongoDB
npm run dev      # http://localhost:5000
```

Frontend:

```
cd client
npm install
# .env: VITE_API_URL=http://localhost:5000/api
npm run dev      # http://localhost:5173
```

If the backend is not running, the frontend still works using *mockData.js*.

8. Current status and observations

Based on reading the code and git state (branch *main*, 3 commits):

- **Uncommitted changes:** (a) *api.js* axios timeout raised from 4 s to 60 s (likely to tolerate a slow/cold hosted backend); (b) *db.js* no longer falls back to *mongodb://127.0.0.1:27017/socialnetworker* — MONGO_URI is now mandatory, otherwise mongoose.connect receives undefined.
- **Security concern:** *client/.env* and *server/.env* are tracked in git. The server .env contains the MongoDB connection string, which may hold credentials. Remove them from git history, rotate the DB password, and add .env to .gitignore.
- **Missing route:** the frontend calls *GET /about* (getAboutContent) but the server has no /api/about route, so it always falls back to mock data.
- **README is outdated:** it says only the Home page is built and mentions .env.example files and a hooks/ folder that do not exist; in reality ~35 pages and 11 API resources are implemented.
- **Contact form:** CORS is open to all origins and there is no rate limiting or spam protection on POST /api/contact.
- **Branding leftovers:** a global rename produced names like *LifeAtsocialnetworker*, and the old name Xilligence remains in a file name.
- **Possible improvements:** admin dashboard for content/messages, email notification on contact submit, input validation (e.g. express-validator), tests, SEO meta tags per page, image optimisation, and a real /locations list page.